Assignment (-2)

Section (A) COA.

Que (1) → Clocking methodology is a synchronization technique where operations occurs based on clock signals, coordinating data movement between registers and functional units

Que (2) → A clock cycle is the smallest time unit of processor operation. It determines how fast instruction are executed and overall CPU speed.

Que (3) → Amdahl's law states that overall speedup is limited by the non-parallelizable portion of a program

$$\boxed{\text{Speed up} = \frac{1}{(1-p) + p/s}}$$

Que (4) → The Data path consists of hardware components like ALU, registers, multiplexers and buses that perform data processing.

Que (5) → The control unit generates control signals to coordinates operations of ALU, registers and memory.

Que 6 → A single-cycle processor executes an entire instruction in one clock cycle, making it simple but resulting in a slower clock speed

Que 7 → A multi-cycle processor divides instruction execution into multiple steps, ~~which~~ with each step executed in a seperate clock cycle, improving efficiency.

Que 8 → Pipelining is a technique where multiple instruction are executed in an overlapping manner by dividing execution into stages

Que 9 → Instruction level parallelism is the ability of a processor to execute multiple instruction simultaneously within a pipeline

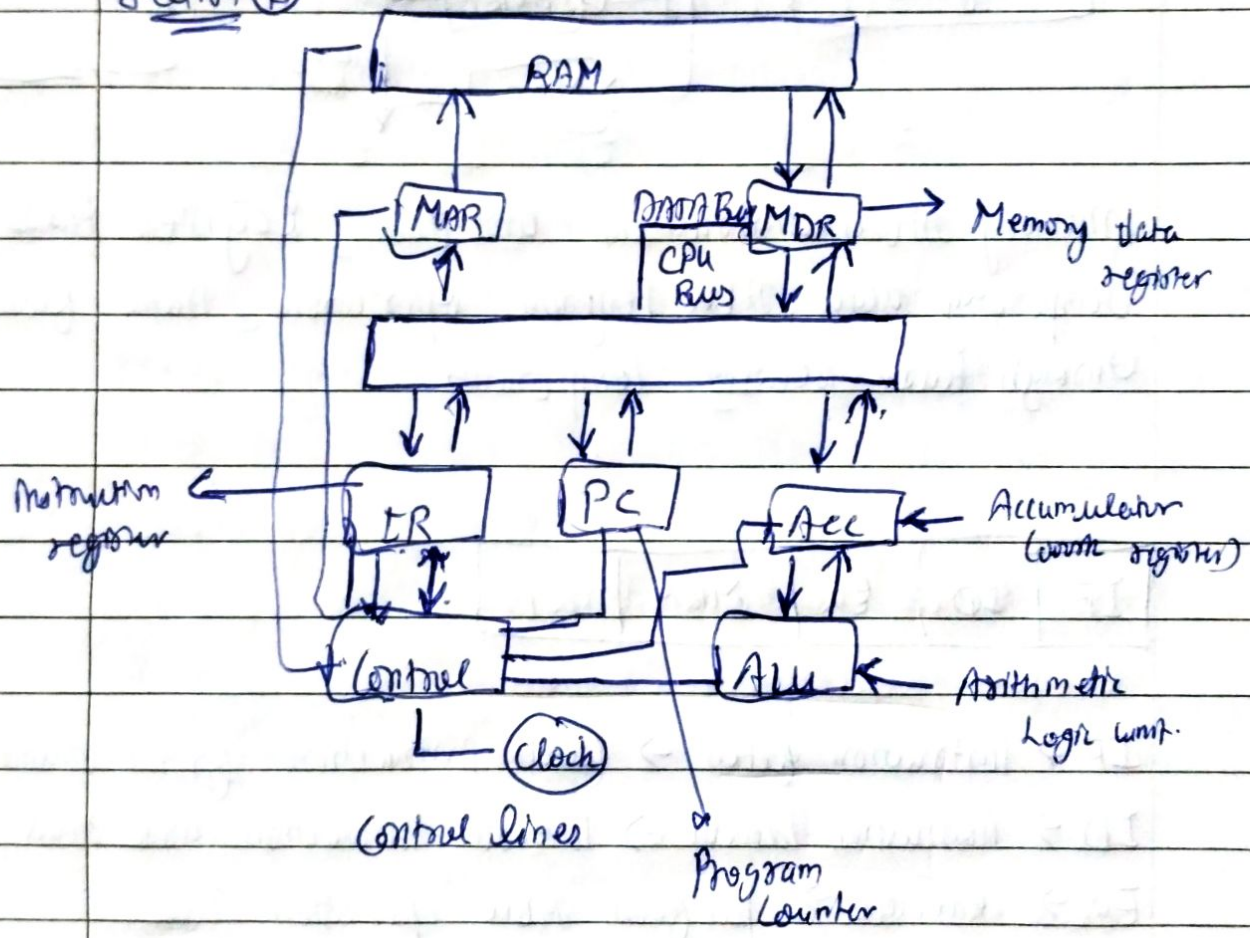
Que 10 → A data hazard occurs when an instruction depends on the result of a previous instruction that has not yet completed

Que 11 → Control hazard occurs due to branch instruction that change the flow of execution, causing uncertainity in the pipeline

Que (12) ⇒ Pipeline hazards can be reduced using forwarding, pipeline stalls, branch prediction and instruction reordering.

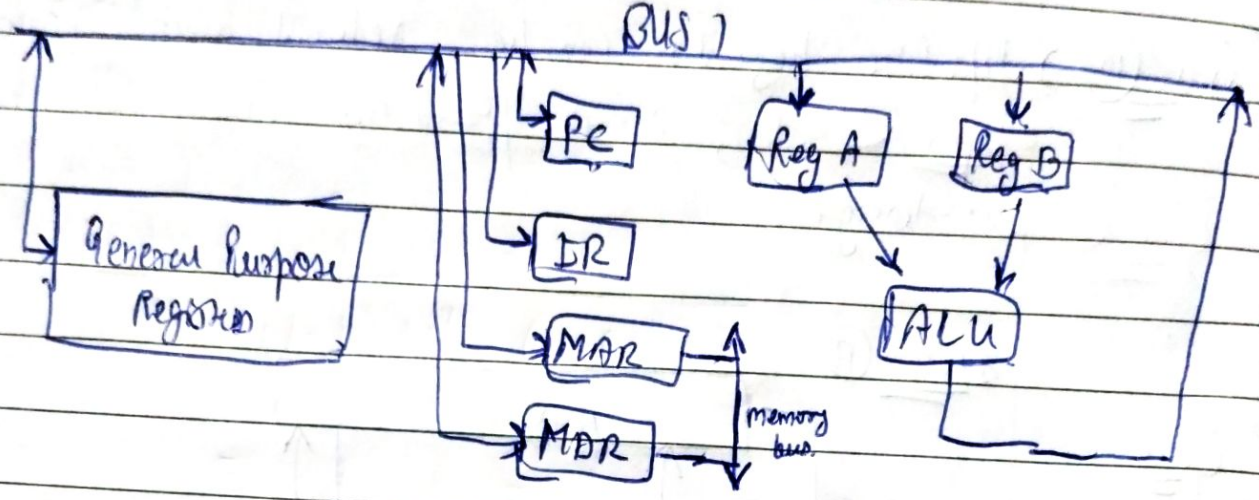
Section (B)

Que




The clock signal controls when data is transferred. Data moves from A → ALU → B, transfer happens only at clock edges, ensures all components work in synchronization

Que →




memory stores instructions and data, Registers holds temporary data, ALU Performs operations, Data flows through buses between components

Que →

| IF | ID | EX | MEM | WB |
|---|---|---|---|---|

IF → Instruction fetch → fetch instruction from memory
ID → Instruction Decode → Decode instruction and read registers
EX → execute → perform ALU operation
MEM → memory Access → Read / write data from / to memory
WB → write back → write result back to registers

| Cycle → | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|
| Inst 1 → | IF | ID | EX | MEM | WB |
| Inst 2 → | | IF | ID | EX | MEM |
| Inst 3 → | | | IF | ID | EX |

__/__/____

Que → Single Cycle Processor

$\boxed{IF + ID + EX + MEM + WB}$ → 1 clock cycle

Multi cycle processor :

$\boxed{IF} \rightarrow \boxed{ID} \rightarrow \boxed{EX} \rightarrow \boxed{MEM} \rightarrow \boxed{WB}$

$C_1 \quad C_2 \quad C_3 \quad C_4 \quad C_5$

Single cycle → entire instruction completes in one long clock cycle, simple design, clock cycle must be long enough for all stages

Multi cycle → instruction divided into multiple stages each stage takes one clock cycle shorter clock cycle better efficiency.

Section ⓒ :→

Que → Frequent data dependencies cause data hazards where an instruction needs the result of a previous instructions that is not yet available.

Solution → forwarding → Directly pass results b/w pipeline stages

pipeline stalls → Delay execution until required data is available

Que → $P = 0.8$

$S = 4$

Speedup → $\frac{1}{(1-p) + P/S} = \frac{1}{(1-0.8) + \frac{0.8}{4}}$

$= \frac{1}{(0.2 + 0.2)} = \frac{1}{0.4} = 2.5$

Speedup = 2.5

Que → A multi cycle processor uses shorter clock cycles and better hardware utilization as each stage executes seperately, improving efficiency.

Que → This causes a control hazard which can be reduced using branch prediction techniques.

Section (D) →

Que → | IF | ID | EX | MEM | WB |

| Cycle | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|
| Int 1 | IF | ID | EX | MEM | WB | — | — | — |
| Int 2 | — | IF | ID | EX | MEM | WB | — | — |
| Int 3 | — | — | IF | ID | EX | MEM | WB | — |
| Int 4 | — | — | — | IF | ID | EX | MEM | WB |

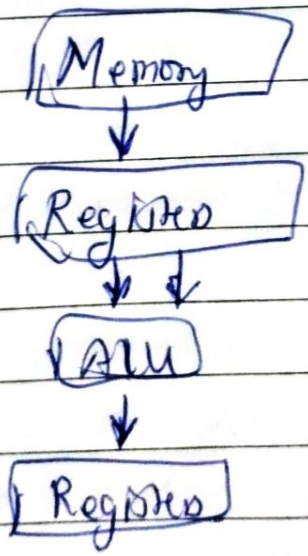
EX → execute

ALU performs operations (Add, Subtract)

Calculate branch target addresses

Performs arithmetic / logic computations

Que →



__/__/__

Memory → Stores instruction and data
Registers → fast storage for operands and results
ALU → Performs arithmetic and logic operations
Buses → Move instructions & data flow